

NEW AUTHENTICATION SYSTEM

CRM-Native Authentication Replacement Specification

Design • Migration • Security Controls • Verification • Rollback • Operations

SECURITY NOTE: Design only. This document does not authorize implementation or production changes. Build and verify the replacement in isolation before removing the existing authentication dependency.

1. Purpose and Decision

The project will replace the current Clerk-backed authentication/session layer with CRM-owned authentication. The existing CRM authorization model is retained. This is an authentication migration, not a redesign of tenant/RBAC/RLS.

AUTHENTICATION

Browser → CRM Auth → secure session → CRM User

AUTHORIZATION

CRM User → tenantId → UserRole → Role → RolePermission
→ server authorization → PostgreSQL RLS/FORCE RLS

Admin and Employee are both CRM identities. Their authority comes from server-side role/permission data, never from passwords, email names, URLs, frontend flags, or client-supplied role values.

2. Current Architecture — Source-Derived Baseline

Component	Current responsibility	Treatment
Clerk	Authentication, external identity, session issuance	REMOVE
CRM User	Business identity, tenant, employee ID, status	RETAIN
UserInvitation	Invitation, tenant, role, department, hashed token, expiry/status	RETAIN + ADAPT
UserRole / Role / Permission / RolePermission	Authorization	RETAIN
tenantId	Server-derived company boundary	RETAIN
RLS/FORCE RLS	Database tenant backstop	RETAIN
DeviceSession	Existing session/device building block	REVIEW + REWORK
Clerk webhook route	External identity synchronization	REMOVE/REPLACE

The reviewed project material explicitly separates authentication from authorization: authentication establishes identity; CRM data establishes tenant, role and permissions; RLS provides a database-level boundary.

3. What We Are Removing

- Clerk middleware as the core authentication gate.
- Clerk auth()/current-user resolution as the source of authenticated identity.
- Clerk frontend SignIn/SignUp/UserButton/useUser dependencies used only for authentication.
- Clerk-specific identity synchronization/provisioning.
- Clerk-specific accept-invite identity verification.
- Clerk-specific user deletion during CRM offboarding.

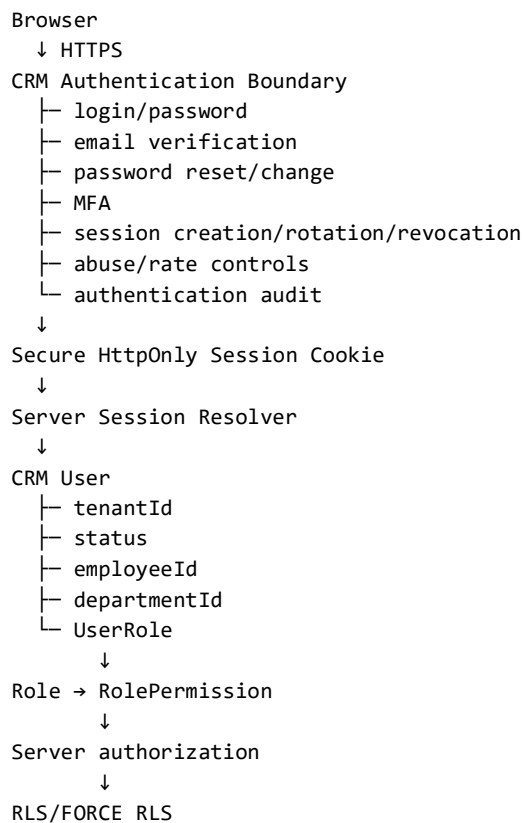
- Clerk identity webhooks whose only purpose is synchronization.
- Production/staging dependence on Clerk keys for core CRM authentication.

4. Why We Are Removing It

- Remove third-party IdP dependence from the core CRM login path.
- Make CRM authentication identities first-class in each isolated deployment.
- Allow each company deployment to own its authentication lifecycle.
- Control invitation, verification, recovery, session revocation and account lifecycle directly.
- Preserve the existing tenant/RBAC/RLS model instead of creating a second authorization system.
- Remove the current Clerk-specific staging/authentication dependency.

SECURITY NOTE: Removing Clerk transfers security responsibility into the CRM. The replacement is acceptable only when the controls in this document are implemented and verified.

5. Target Architecture



6. Identity and Database Model

Field/object	Purpose	Security rule
User.id	Stable internal identity	Identifier, never a secret
User.email	Login/recovery identity	Normalized; uniqueness policy explicit
User.passwordHash	Password verifier	Argon2id-class password hashing; never plaintext
User.passwordChangedAt	Credential lifecycle	Used for session invalidation policy
User.emailVerifiedAt	Email ownership state	Server-controlled

User.status	INVITED/ACTIVE/INACTIVE/LOCKED/etc.	Checked during authentication
User.tenantId	Tenant membership	Never client-selected
User.employeeId	Business employee number	Not an authentication credential
AuthSession	Session record	Store only token hash; expiry/revocation metadata
AuthToken	Reset/invite/verification token	Store only hash; purpose/expiry/single use
MfaFactor	MFA metadata	Protected secret handling
RecoveryCode	MFA recovery	Store hashes; one-time use
SecurityEvent	Authentication audit	Never store credentials/tokens

7. Password System

- Use a modern memory-hard password hashing scheme such as Argon2id with reviewed parameters.
- Never store plaintext passwords or reversible password encryption.
- Never log passwords, reset tokens, invitation tokens, MFA codes or session tokens.
- Use unique salts through the password hashing library.
- Define a deliberate password policy; avoid arbitrary complexity rules that encourage predictable passwords.
- Consider breached/common-password screening for new and changed passwords.
- Use a server-side pepper only if key management and rotation are explicitly designed.
- Password changes must trigger the documented session invalidation/rotation policy.
- Authentication failures should not reveal whether an account exists.

8. Session Security

```

Login
↓
Generate high-entropy random session secret
↓
Store only a cryptographic hash of it
↓
Secure + HttpOnly + deliberate SameSite cookie
↓
Session resolver → ACTIVE CRM User
↓
tenant/RBAC authorization

```

- Prefer opaque random server-side sessions for browser authentication.
- Store only a hash of the session token in the database.
- Use Secure cookies in production.
- Use HttpOnly cookies.
- Set SameSite deliberately for the deployment.
- Never put session identifiers in URLs.
- Never use localStorage as the primary session-token store.
- Implement absolute expiry and idle timeout.
- Track createdAt, lastSeenAt, expiresAt and revokedAt.
- Support per-device session revocation and global logout.
- Rotate the session after successful authentication and other security-sensitive transitions.
- Do not let stale cache preserve authorization after revocation.

9. Session Fixation / Revocation / Disablement

- A pre-login session must never become the authenticated session.
- Successful login creates a fresh session.
- Password reset/change, MFA changes and sensitive recovery operations follow explicit session invalidation rules.
- Logout revokes the server-side session; deleting a cookie alone is insufficient.
- Global logout revokes every active session.
- Disabling an Employee must stop an old session from authorizing subsequent protected requests.
- Session validity must be checked on each protected request or through an equivalently strong mechanism.

10. CSRF Protection

- Cookie-authenticated state-changing requests require an explicit CSRF defense.
- SameSite is an additional control, not the sole design assumption.
- Use Origin/Referer checks where appropriate.
- Use CSRF tokens for applicable browser state changes.
- Do not disable CSRF merely because requests originate from the CRM UI.
- Machine webhooks are separate: they require signature, timestamp, replay and idempotency controls.

11. Password Reset

- Generate cryptographically random reset tokens.
- Store only token hashes.
- Bind tokens to user and purpose.
- Use short expiry and single-use semantics.
- Consume tokens atomically to prevent concurrent reuse.
- Rate-limit reset requests and verification attempts.
- Use generic public responses to avoid account enumeration.
- After reset, revoke/rotate relevant sessions.
- Never log or expose reset secrets.

12. Email Verification

- Verification tokens are random, hashed at rest, purpose-bound, expiring and single-use.
- Verification is tied to the intended account, not just an email string from the browser.
- Email changes require re-verification and, for sensitive changes, re-authentication.
- Resend is rate-limited.
- Do not create account-enumeration leaks.

13. MFA and Recovery

- Support TOTP as a baseline; evaluate WebAuthn/passkeys for stronger phishing resistance.
- Protect TOTP secrets appropriately; never log them.
- Require recent authentication for MFA enrollment/removal.
- Recovery codes are stored only as hashes and are single-use.
- MFA recovery must not become a hidden privilege-escalation path.

- Changing/removing MFA generates an audit event and follows session policy.
- Define recovery procedures before production rollout.

14. Login Abuse Controls

- Rate-limit login attempts per account identifier and source/network identity.
- Protect against distributed credential stuffing, not only one-IP brute force.
- Use progressive backoff or carefully designed temporary lockout; avoid attacker-controlled permanent lockout.
- Rate-limit password reset, verification resend, MFA, invitation redemption and recovery separately.
- Do not allow attacker-controlled arbitrary rate-limit keys.
- Monitor failed-login spikes and suspicious recovery patterns.

15. Authentication Endpoint Contract

Endpoint	Mandatory controls
POST /login	Rate limit; generic failure; password verification; fresh session; audit event
POST /logout	Server-side revoke; cookie clear; idempotent
POST /logout-all	Revoke all sessions; step-up policy where appropriate
POST /password/change	Authenticated + recent-auth/step-up; session policy
POST /password/reset/request	Generic response; rate limit; hashed token
POST /password/reset/confirm	Hashed single-use token; expiry; session invalidation
POST /email/verify	Hashed single-use token; expiry; account binding
POST /mfa/verify	Attempt limit; factor binding; audit
POST /mfa/enroll	Recent auth; secure factor setup; confirmation
POST /invite/accept	Single-use invitation; tenant/role from server; atomic redemption

16. Employee Invitation and Provisioning

```

Admin authenticated
↓
Resolve Admin CRM User + tenant
↓
Check USER/CREATE permission
↓
Create UserInvitation
  email / tenantId / roleId / departmentId
  hashed token / expiry / inviter
↓
Employee opens invitation
↓
Set password + verify email + enroll MFA as required
↓
Atomic redemption
↓
CRM User ACTIVE
↓
UserRole assigned from invitation state
↓
Secure session created

```

- Browser cannot choose tenantId, roleId, departmentId or permission set.

- Invitation tokens are hashed, expiring and single-use.
- Concurrent redemption is prevented transactionally.
- Expired/revoked invitations fail.
- An old invitation must not silently restore an obsolete role.
- Employee activation occurs only after all required verification conditions are met.

17. Authorization Is Preserved

```

Session
↓
CRM User
↓
tenantId
↓
UserRole
↓
Role
↓
RolePermission
↓
requireAuth / requirePermission / withTenant
↓
RLS/FORCE RLS

```

- Never trust role from the browser.
- Never trust tenantId from the browser.
- Never treat a userId supplied by the browser as proof of identity.
- Keep server-side authorization on every privileged action.
- Keep RLS/FORCE RLS as the database backstop.
- Do not create a second, conflicting authorization system inside authentication.

18. Admin vs Employee Security Tests

Attack/action	Required result
Employee opens /admin	Server-side denial
Employee calls Admin-only API directly	Server-side permission denial
Employee changes role in payload	Rejected/ignored; role comes from server
Employee changes tenantId in payload	Rejected/ignored; tenant comes from server
Employee supplies another user ID	No privilege escalation or cross-tenant access
Employee disabled while session exists	Existing session cannot authorize
Logout then reuse old cookie	Authorization fails
Password reset then reuse old session	According to policy, old session revoked
Invitation replay	Rejected
Reset-token replay	Rejected
MFA recovery abuse	Rejected/contained

19. Tenant Isolation

- Tenant context is derived from the authenticated CRM User.
- Client-supplied tenantId can never override it.
- Tenant-scoped queries/writes retain the boundary.
- Cross-tenant object IDs fail without leaking existence.
- RLS/FORCE RLS remains enabled.

- System/bypass DB roles are restricted to explicitly authorized internal operations.
- Background jobs carry authoritative tenant context.

20. Audit Logging

- Log login success/failure, logout, logout-all, password reset, email verification, MFA changes, invitation redemption, session creation/revocation, account lock/unlock and security-sensitive changes.
- Record safe actor/user, tenant, event type, timestamp and correlation ID as appropriate.
- Never log passwords, tokens, cookies, authorization headers or secret keys.
- Restrict audit mutation/deletion.
- Define retention and privacy before production.

21. Secrets and Key Management

- Authentication encryption/signing keys belong in the deployment secret store.
- Never commit secrets.
- Never print secrets to terminals, logs, reports, screenshots or temporary files.
- Never copy Production secrets into local, preview, staging or test environments.
- Keep authentication secrets separate from database/provider/webhook secrets.
- Design key rotation before implementation.
- If a secret reaches an uncontrolled location, treat it as compromised and rotate it.

22. Error, Timing and Enumeration Security

- Use generic public authentication failures.
- Do not expose whether an account exists through login/reset/verification responses.
- Where appropriate, perform comparable work for unknown users to reduce timing leaks.
- Do not expose stack traces, ORM errors, token values or internal authorization details.
- Keep detailed diagnostics server-side.

23. Race Conditions and Atomic Security Operations

- Invitation redemption must be atomic.
- Reset-token consumption must be atomic.
- Recovery-code consumption must be atomic.
- Login lockout counters must handle concurrency.
- Disable-user and session authorization must have explicit consistency semantics.
- Email change/verification must reject stale tokens.
- Use database constraints and transactions for security invariants.

24. Sensitive Changes / Step-Up Authentication

- Require recent authentication for sensitive account changes.
- Require MFA/step-up where risk policy requires it.
- Role changes, user disablement, MFA changes, recovery-email changes and security settings are sensitive operations.
- Privilege reduction must not be defeated by stale authorization caches.

25. Removal of Clerk-Specific Components

- Remove Clerk middleware only after native session middleware is verified.
- Replace Clerk auth resolution with native session resolution.
- Replace Clerk UI with native authentication UI.
- Replace Clerk accept-invite identity verification with native verification.
- Remove Clerk user lifecycle synchronization after equivalent CRM lifecycle is verified.
- Remove Clerk webhooks after no runtime path depends on them.
- Remove Clerk environment variables only after deployment dependency scans show no remaining runtime use.

26. Migration Strategy — Do Not Big-Bang Production

Phase	Work	Exit evidence
A	Freeze design + threat model	Approved design and threat model
B	Build auth core	Unit/integration security tests
C	Build reset/verification/invitation/MFA	Replay/race/abuse tests
D	Connect native sessions to existing RBAC/RLS	Tenant/RBAC tests
E	Deploy isolated staging	Six isolation gates pass
F	Run Employee workflow	Runtime evidence
G	Re-run Admin workflow	Runtime evidence
H	Run adversarial authentication suite	Evidence-backed results
I	Controlled production cutover	Rollback tested + approval
J	Remove Clerk completely	Dependency scan + runtime verification

27. Clerk Identity Migration Rules

- Do not import or reverse-engineer Clerk passwords.
- Do not invent passwords for existing users.
- Require users to establish native credentials through a controlled migration/reset flow.
- Do not attach a native account based solely on an unverified email.
- Preserve CRM User.id, tenant, employeeid, roles, permissions and business history.
- Retire clerkId only after native identity is established and rollback conditions are satisfied.
- Record migration state for every affected account.

28. Rollback

- Take verified backups/snapshots before identity migration.
- Test rollback in isolated staging first.
- Define reversible database changes.
- Define native-session revocation during rollback.
- Define how users return to the prior authentication state without altering business authorization.
- Keep a documented rollback window and decision authority.
- Never require production secrets to be copied into local machines for rollback.

29. AI-Agent / Automation Anti-Forgetting Checklist

These are explicit guardrails for future AI coding agents because authentication failures frequently occur in the details rather than the visible login screen.

- Never store plaintext passwords.
- Never use MD5/SHA-1/SHA-256 alone as a password hash.
- Never invent a homemade password hash.
- Never put primary auth tokens in localStorage.
- Never put session tokens in URLs.
- Never log passwords, cookies, Authorization headers, reset tokens, invite tokens or MFA codes.
- Never trust client userId, tenantId, role, permission list or status.
- Never rely on UI hiding for authorization.
- Never create a hidden Admin/test backdoor.
- Never make a test bypass reachable in Production.
- Never disable RLS to make a test pass.
- Never use a Production database for destructive auth testing.
- Never copy Production credentials into local/staging/test.
- Never create hardcoded test passwords in source.
- Never use predictable session IDs.
- Never reuse a pre-login session after authentication.
- Never forget absolute expiry and idle timeout.
- Never forget global logout.
- Never forget stale-session behavior after disablement.
- Never forget invitation/reset/MFA replay protection.
- Never let recovery bypass normal authorization.
- Never let password reset grant a role.
- Never let email change silently transfer account ownership.
- Never expose account existence unnecessarily.
- Never make security decisions only in client components.
- Never silently bypass rate limiting because Redis is unavailable.
- Never silently accept stale cached permissions.
- Never forget database uniqueness/transaction constraints for security invariants.
- Never expose ORM/database errors to users.
- Never mark PASS from source inspection when runtime evidence is required.

30. Threat Model

Threat	Primary controls	Evidence required
Credential stuffing	Rate limits, password security	Abuse tests
Brute force	Backoff/attempt controls	Negative tests
Session theft	HttpOnly/Secure, revocation	Session tests
Session fixation	Rotation on login	Dedicated test
CSRF	CSRF defense + SameSite/Origin	Negative tests
XSS token theft	HttpOnly + CSP + output safety	Security-header tests
Reset takeover	Hashed, expiring, single-use tokens	Replay/race tests
MFA bypass	Step-up + secure recovery	Adversarial tests
Privilege escalation	Server RBAC	Direct API tests
Tenant escape	Derived tenant + RLS	Cross-tenant tests
IDOR	Ownership/tenant checks	Object substitution
Mass assignment	Strict schemas/allowlists	Payload mutation

Invite takeover	Bound hashed single-use invite	Wrong-account/replay tests
DB compromise	Hashed passwords/session tokens	Exposure review
Audit tampering	Restricted audit writes	Audit security review

31. Availability and Failure Semantics

- If the auth database is unavailable, do not authenticate from stale browser state.
- If Redis is unavailable, do not bypass security-critical revocation/rate controls simply to keep login working.
- Internal logs may distinguish invalid credentials from infrastructure failure; public responses should remain appropriately generic.
- Never create a successful session when authoritative user/status validation cannot complete.
- Protected operations fail closed when authentication cannot be established.

32. Privacy and Data Minimization

- Collect only authentication metadata needed for security and operations.
- Minimize IP/device retention and define retention.
- Never send passwords, MFA codes or session secrets to analytics or AI systems.
- Restrict access to security telemetry.

33. Performance and Scalability

- Index login/session lookup fields.
- Keep session validation efficient without weakening revocation.
- Bound rate-limit cardinality and expiry.
- Measure password verification cost under expected concurrency.
- Measure login/session/authorization latency and DB connection behavior.
- Repeat high-concurrency tests on the intended deployment topology.

34. Implementation Rules for AI Coding Agents

- Read this document and the current Prisma/auth/RBAC/RLS implementation before editing.
- Inventory every Clerk dependency before deleting any.
- Create a native auth abstraction before migrating consumers.
- Do not mix authentication migration with unrelated CRM feature work.
- Turn each security invariant into an explicit test.
- Keep an issue register: issue, severity, evidence, fix, verification, status.
- Use isolated test data/infrastructure.
- Never request or print production secrets.
- Do not delete old authentication code until replacement verification passes.

35. Definition of Done

- No production runtime path requires Clerk.
- Native login is implemented and independently tested.
- Password storage uses reviewed memory-hard hashing with no plaintext.

- Session creation, rotation, expiry, idle timeout, logout and revocation are verified.
- Disabled users cannot continue using stale sessions.
- Password reset is secure, expiring, single-use and session-safe.
- Email verification is secure and single-use.
- MFA and secure recovery are implemented to the agreed level or explicitly deferred with documented risk.
- CSRF protection is implemented and tested.
- Login/reset/MFA/invitation abuse controls are implemented and tested.
- Admin/Employee authorization remains server-side.
- Tenant context remains server-derived.
- RLS/FORCE RLS remains enabled and verified.
- IDOR and mass-assignment tests pass.
- Invitation redemption is atomic and replay-safe.
- Authentication audit logs contain no secrets.
- Secrets are stored only through approved secret management.
- Staging and Production authentication resources are isolated.
- Rollback has been tested.
- Admin and Employee workflows are runtime-verified.
- Clerk-specific code is removed only after dependency closure is proven.

36. Current Project Boundary

Item	Status
Clerk removal	PLANNED — NOT IMPLEMENTED
CRM-native authentication	DESIGN SPECIFICATION
Password/session implementation	NOT IMPLEMENTED
MFA	DESIGN REQUIREMENT — implementation pending
Employee runtime	Currently blocked by authenticated test environment
RBAC-01	Code fix committed as 69174e2; runtime pending
PriceBook uniqueness	Uncommitted / blocked
Production certification	Not complete

37. Source Basis

This document uses the project repository and project documents as the baseline, including:

- Current auth.ts, middleware.ts, user.service.ts, provisioning service, accept-invite route and Prisma schema.
- Project Context Review.txt.
- Important Doubts.docx.
- How to handle credentials.docx.
- AUTHENTICATED_ADMIN_EMPLOYEE_TEST_PLAN.md.
- PHASE_15B_CLERK_DEVELOPMENT_VERCEL_DEMO_AUDIT.md.
- PHASE_15B_EMPLOYEE_INVITATION_AUTHORIZATION_AUDIT.md.
- S17-B2_STAGING_INFRASTRUCTURE_PROVISIONING_PLAN.md.
- S17-C_PRODUCTION_READ_ONLY_CERTIFICATION.md.
- Current project handovers, progress reports and Employee source audits.

Source-derived facts describe the current system. Security requirements added in this specification are proposed requirements for the replacement and must not be mistaken for capabilities already implemented.

38. Final Principle

Authentication proves who the user is. Authorization decides what that identity may do. Tenant isolation decides whose data it may touch. RLS provides a database backstop.

The objective is not merely to replace Clerk with a login page. The objective is to own the complete authentication lifecycle without weakening password security, sessions, recovery, MFA, CSRF, abuse resistance, auditability, tenant isolation, RBAC, RLS, or operational safety.